AUDITORIA DE SEGURANÇA

Relatório de Auditoria de Segurança — Distributed Wagering Processor

Data: 01 de novembro de 2026

Situação após correções em 01 de novembro de 2026

Escopo

Auditoria da aplicação em `app/`, com dois auditores independentes (A e B) cujos achados foram consolidados neste relatório.

Auditor A	TypeScript estrito (Bun 1.4.4) · NestJS 11 (Express), helmet 8.3.0, jose 6.2.15 · MikroORM 6.6.17 com SQL explícito parametrizado (pg) Autenticação: OIDC/JWT opcional (AUTH_MODE=oidc): AuthGuard + OidcTokenVerifier (JWKS remoto) + ClaimsProviderIdentity · Front-end: Nenhum (apresentação HTML estática em docs/apresentacao; Swagger UI em /docs) Deploy: Dockerfile oven/bun:1.4.4-alpine, USER bun, docker-compose (postgres, ministack, app, prometheus, jaeger, grafana), GitHub Actions CI
Auditor B	TypeScript estrito sobre Bun 1.4.4 · NestJS 11 (platform-express), helmet 8.3.0, zod 4 · MikroORM 6.6.17 com SQL explícito sobre pg Autenticação: OIDC/JWT opcional (jose 6.2.15; AUTH_MODE=disabled|oidc), escopo wallet:admin e claim provider_id · Front-end: Apresentação HTML estática (docs/apresentacao/index.html) e Swagger UI em /docs; sem SPA Deploy: docker-compose (postgres 17.6, MiniStack/SQS, app, prometheus, jaeger, grafana), Dockerfile oven/bun:1.4.4-alpine, USER bun, GitHub Actions por SHA

Nota metodológica

Categoria 1 — Banco sem tranca

Auditor A: Banco sem tranca vira isolamento por provider/tenant. Rodada 2: reverificada a correção de A-02 (claim provider_id) em todos os caminhos que agem em nome de um provider (POST de aposta, GET por id interno, GET por id externo), mais replay de idempotency key, EXTERNAL_ID_CONFLICT, mensagens de erro e eventos. Testado com requisições reais contra o app em AUTH_MODE=oidc (porta 4501, banco wagering_bigorna), com tokens de dois providers e de um admin assinados por um IdP/JWKS local meu (jose).

Auditor B: Fora do escopo do Auditor B (categoria do Auditor A).

Categoria 2 — Permissão definida no navegador

Auditor A: Não se aplica: não há frontend com papéis, flags ou localStorage. O Swagger UI não toma decisão de permissão; toda decisão está no servidor (AuthGuard e ProviderIdentityPort).

Auditor B: Fora do escopo do Auditor B (categoria do Auditor A).

Categoria 3 — IDOR

Auditor A: IDOR: percorridos todos os handlers de src/interfaces/http/*.controller.ts (11 rotas mais /docs). Para cada rota foi montada a tabela auth esperada x implementada (ver coverage) e testada com 3 identidades (sem token, provider A, admin). Conferidas a ordem guard/pipe (401 antes de 400/404) e a diferença 404 x 403 entre providers.

Auditor B: Fora do escopo do Auditor B (categoria do Auditor A). Só li o AuthGuard e os controllers para confirmar o que fica sem guarda (/metrics, /health).

Categoria 4 — Chaves expostas

Auditor A: Fora do escopo do Auditor A (Auditor B). Só se verificou que o token não aparece nos logs.

Auditor B: Reverifiquei B-01 a B-05 e B-09 lendo docker-compose.yml, init-roles.sql, config.ts, mikro-orm.config.ts, Dockerfile, .dockerignore, .gitignore, .env.example e README. Testei a guarda de produção do config.ts com 21 variações via loadConfig (URL-encoding, usuário diferente, ?password=, esquema postgresql, ' test', 'TEST', NODE_ENV=Production/prod/'production ', JWKS http, FAULT_INJECT) e subi o app de verdade (TEST_DB_NAME=wagering_forja, portas 4101/4102/4103 de TEST_PORT_BASE): dev, produção com senha '%61pp' e produção com AUTH_MODE=oidc. Histórico: git log --all -p -G com padrões de AKIA/ASIA, PRIVATE KEY, ghp_/github_pat_/xox/sk-, JWT e JWK ('kty'), git log -S'-----BEGIN', 'git log --all --name-only' por .env/.pem/.key, e git grep de senhas e URLs com credencial no HEAD (inclui test/, scripts/, docs/). O banco wagering_forja foi criado e removido por mim; a infra compartilhada não foi alterada.

Categoria 5 — XSS

Auditor A: Fora do escopo do Auditor A. Não há HTML dinâmico com input do usuário.

Auditor B: Reverifiquei B-10 com requisições reais: headers de /health/live e /docs, 404 com <script> no path, JSON malformado, content-type text/html, body de 70 KB, 401 com cinco formas de Authorization. Li o HTML do /docs por tags script/link/style. Na apresentação, busquei sinks (innerHTML, eval, setAttribute, location, data-tip) e li o JS da tooltip. No gerador de relatório, copiei o script e os fixtures para um diretório temporário fora do repositório, injetei <script> e <img onerror> em todos os campos de texto e rodei com --sample (o HTML foi escrito antes de a etapa do PDF falhar de propósito com EDGE_PATH inexistente), depois contei as tags no HTML.

Categoria extra — Testes extras

Auditor A: Ataques à validação JWT (alg none, HS256 com a chave pública em PEM, kid desconhecido, chave trocada, ES256 contra kid RSA, issuer/audience errados, audience em array, token sem exp, exp expirado por 1s e por 30s, nbf futuro, scope como array/string/scp, scope com prefixo parecido, provider_id de 8 tipos); cabeçalho Authorization malformado (9 variações e 2 cabeçalhos duplicados em socket cru); cache e rotação do JWKS; IdP fora do ar e IdP pendurado; SSRF/esquema em OIDC_JWKS_URL (loadConfig); body limit com JSON, text/plain, form, sem content-type, 70 KB, 30 mil níveis de aninhamento, gzip bomb real (20 MB e 200 MB descomprimidos) e gzip inválido; headers, CORS e preflight; /health/ready; SQL injection no código novo; vazamento do token em log (LOG_LEVEL=debug).

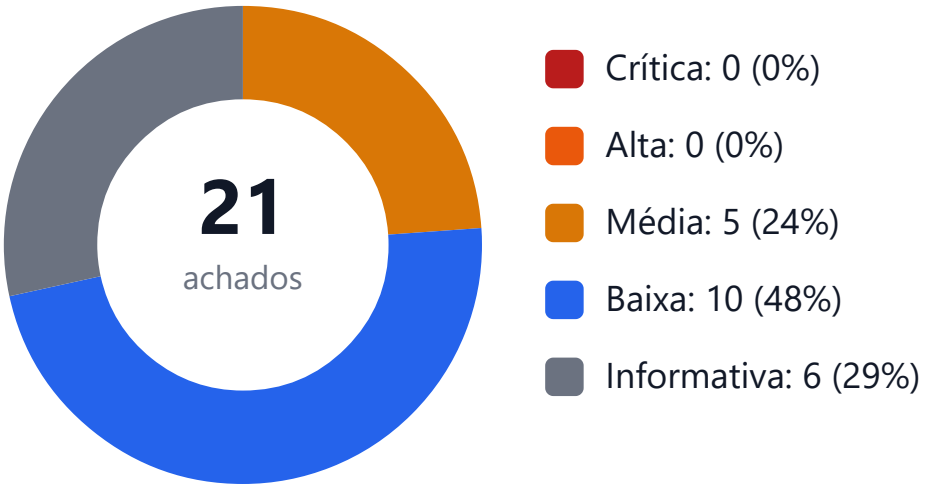
Auditor B: Supply chain: bun audit (0 vulnerabilidades); overrides conferidos no registro npm e no bun.lock; SHAs das actions conferidos com git ls-remote contra as tags v4.4.0 e v2.2.0; manifest de cada imagem do compose e do Dockerfile (docker manifest inspect); USER e COPY do Dockerfile; ignore dos arquivos .env.* com git check-ignore. Os apps de teste foram encerrados no fim e o banco wagering_forja foi apagado.

Resumo executivo

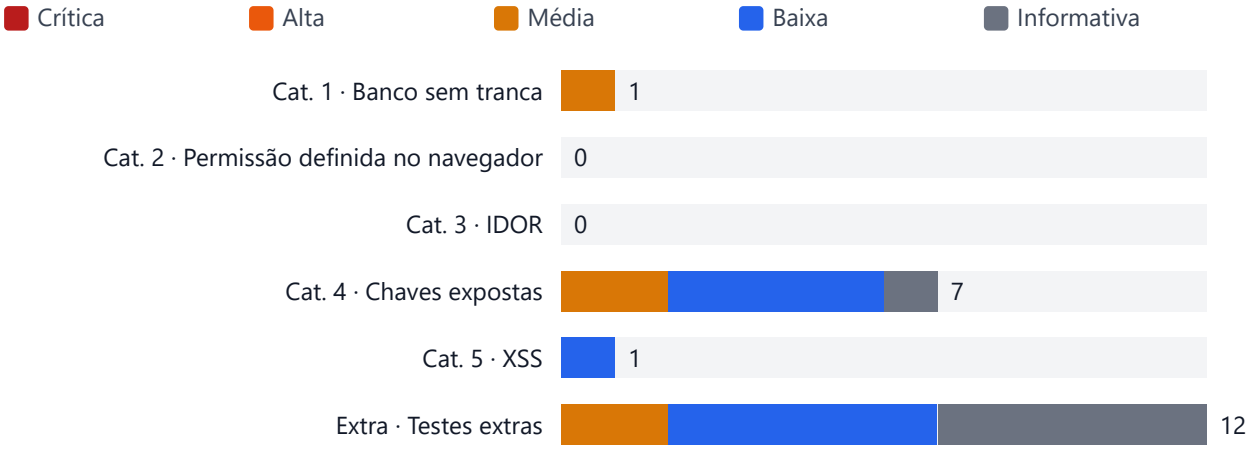
Foram registrados **21** achados (0 crítico(s), 0 alto(s), 5 médio(s), 10 baixo(s), 6 informativo(s)) e **40** pontos fortes. Não há achados críticos ou altos.

0	0	5	10	6	40
Crítica	Alta	Média	Baixa	Informativa	Pontos fortes

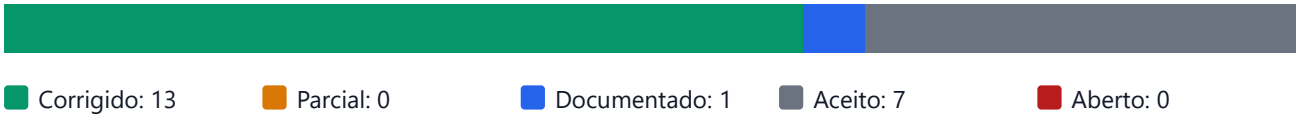
Achados por severidade



Achados por categoria, empilhados por severidade



Achados por status



13 corrigidos, 0 parciais, 1 documentados, 7 aceitos, 0 abertos

Auditor A: 8 achado(s). Auditor B: 13 achado(s).

Pontos fortes

Tipo	Categoria	O que está protegido	Evidência
Ponto forte	Banco sem tranca	Reverifica A-02 (parcial, ver A-R2-01 para o resíduo): o claim provider_id é checado em TODOS os três caminhos que agem em nome de um provider. Token do prov-b com providerId 'prov-a' no body: 403 PROVIDER_MISMATCH; GET por id externo de outro provider: 403 PROVIDER_MISMATCH; GET por id interno de tx de outro provider: 404 idêntico ao inexistente.	`src/interfaces/http/wagering.controller.ts:97,118,133; src/interfaces/http/auth.guard.ts:84-93`
Ponto forte	Banco sem tranca	Idempotência e conflito de id externo são escopados por provider: o mesmo Idempotency-Key e o mesmo externalTransactionId usados pelo prov-b não colidem com os do prov-a (200 PROCESSED nas duas contas); replay com a key de outro provider não é possível porque o providerId do body precisa bater com o token (403). EXTERNAL_ID_CONFLICT e IDEMPOTENCY_CONFLICT só aparecem dentro do mesmo provider, e a mensagem de erro é fixa, sem ecoar dados de outro provider.	`src/application/wagering/submit-wager-transaction.ts:112,121; src/infrastructure/persistence/repositories/wager-transaction.repository.ts:73-78; src/interfaces/http/wagering.controller.ts:29-32`
Ponto forte	Banco sem tranca	Eventos de rejeição carregam só providerId/externalTransactionId/walletId/kind/failureCode do próprio comando, não dados de outro provider; a referência (REFUND/ROLLBACK) é resolvida com findByExternalId(tx.providerId, ...) e markDependentsDue filtra por provider_id.	`src/application/wagering/submit-wager-transaction.ts:149-167; src/application/wagering/wager-engine.ts:153; src/infrastructure/persistence/repositories/wager-transaction.repository.ts:91-96`

Tipo	Categoria	O que está protegido	Evidência
Ponto forte	Banco sem tranca	Reverifica A-01 e A-03: com AUTH_MODE=oidc todas as rotas de negócio exigem token (401 sem token, em 12 rotas testadas) e as de wallet exigem o scope wallet:admin (provider comum: 403 FORBIDDEN_SCOPE). O guard roda antes dos pipes: POST sem token e com corpo inválido ou de 70 KB-JSON responde 401/413 sem vazar validação (o 413 vem do parser, ver A-R2-04).	`src/interfaces/http/wallets.controller.ts:33-36; src/interfaces/http/wagering.controller.ts:54-56; src/interfaces/http/auth.guard.ts:35-53`
Ponto forte	Permissão definida no navegador	Não se aplica e continua assim: não há frontend com papéis, flags ou storage; as decisões de permissão (scope, provider_id) são todas do servidor.	`src/interfaces/http/auth.guard.ts; docs/apresentacao/index.html (estático)`
Ponto forte	IDOR	Tabela de IDOR sem divergência entre auth esperada e implementada nas 11 rotas; nenhuma rota de negócio sem guard; nenhum 404 x 403 que revele a existência de transação de outro provider (rota por id interno responde 404 igual; rota por id externo responde 403 igual para existente e inexistente).	`src/interfaces/http/wagering.controller.ts:106-136; src/interfaces/http/wallets.controller.ts:33-130`
Ponto forte	IDOR	Um token que tem provider_id E wallet:admin age como admin em /wallets e como o provider dele nas rotas de aposta (leu a própria tx: 200), mas não ganha acesso a transações de outros providers; um token só admin não lê transação nenhuma (404).	`src/interfaces/http/auth.guard.ts:70-93; teste dinâmico 'admin+provider_id'`
Ponto forte	Testes extras	Confusão de algoritmo bloqueada: alg none (jose UnsecuredJWT e montado à mão), HS256 assinado com a chave pública em PEM, ES256 contra kid RSA, kid desconhecido e chave trocada com o kid correto resultam todos em 401. A lista algorithms [RS256, ES256] é aplicada.	`src/infrastructure/auth/oidc-token-verifier.ts:22-26`

Tipo	Categoria	O que está protegido	Evidência
Ponto forte	Testes extras	Claims validados com tipo estrito: provider_id número, array, string vazia, null, objeto, ausente, com espaço ou em outra caixa resulta em 403 PROVIDER_MISMATCH (nunca compara com coerção); scope como array ou claim scp não concede nada (403); 'wallet:admin2' e 'xwallet:admin' não concedem (comparação por token inteiro); audience em array que contém a esperada é aceita (correto); issuer e audience errados: 401; exp 1 s e 30 s no passado: 401 (clockTolerance zero); nbf futuro: 401.	`src/infrastructure/auth/oidc-token-verifier.ts:27-29; src/interfaces/http/auth.guard.ts:49-52,84-86`
Ponto forte	Testes extras	Cabeçalho Authorization estrito: 'bearer' em minúsculas e espaço final aceitos (inofensivo), mas espaço duplo, dois tokens, 'Basic', sem esquema, só 'Bearer', tabulação e lista com vírgula dão 401. Com dois cabeçalhos Authorization em socket cru o Node usa o primeiro (válido+lixo: 200; lixo+válido: 401), sem confusão entre eles.	`src/interfaces/http/auth.guard.ts:39-40`
Ponto forte	Testes extras	Fail-closed com IdP indisponível: JWKS inalcançável ou pendurado nunca deixa passar (sempre 401, ver A-R2-03 para o código de status). O motivo da falha nunca chega ao cliente: corpo fixo 'missing or invalid credentials' para todas as causas.	`src/interfaces/http/auth.guard.ts:42-47`
Ponto forte	Testes extras	O token não vai para o log: com LOG_LEVEL=debug, 94 linhas de log (inclusive o erro 500 gerado) não contêm o JWT, nem a assinatura, nem a palavra 'authorization'. O logger HTTP tem autoLogging desligado e o filtro de exceções só loga nome e stack do erro.	`src/app.module.ts:LoggerModule.forRoot autoLogging:false; src/interfaces/http/http-error.filter.ts:84-91`
Ponto forte	Testes extras	Reverifica A-05 (body): limite de 64 KB aplicado e devolvido como 413 no formato padrão, inclusive antes da autenticação; gzip bomb real (20 MB e 200 MB zerados, 19 KB e 194 KB comprimidos) retorna 413 em ms sem derrubar o processo; JSON com 30 mil níveis de aninhamento: 400 sem estourar a pilha. Content-Type text/plain, application/x-www-form-urlencoded e ausente: o corpo não é parseado e a validação responde 400. Rate limit continua ausente no app (aceito: ARCHITECTURE §16 delega à borda).	`src/main.ts:12,22-34,74-75`
Ponto forte	Testes extras	Reverifica A-06: helmet ativo (CSP default-src 'self' e nosniff observados), X-Powered-By desligado no código, CSP removido só em /docs (startsWith('/docs'), e /docsx dá 404 sem CSP porque a rota não existe). CORS não habilitado: preflight OPTIONS retorna 404 sem Access-Control-Allow-Origin e GET com Origin não recebe ACAO.	`src/main.ts:16-24,72-73`

Tipo	Categoria	O que está protegido	Evidência
Ponto forte	Testes extras	Reverifica A-04: reconciliação (agregação sobre todo o ledger com FOR SHARE) agora exige wallet:admin (anônimo 401, provider 403), reduzindo a amplificação a quem tem token admin.	`src/interfaces/http/wallets.controller.ts:33-36,111-130; src/application/wallet/reconcile-wallet.ts:26-33`
Ponto forte	Testes extras	Reverifica A-08: SET LOCAL lock_timeout continua interpolado, mas o valor vem só do construtor (nunca de requisição) e é validado por /^\d+(ms|s)$/ antes de qualquer uso; nenhum dado de requisição chega a SQL por concatenação no código novo (auth e config não tocam SQL).	`src/infrastructure/persistence/mikro-unit-of-work.ts:16,24-26,36`
Ponto forte	Testes extras	Reverifica A-09: X-Correlation-Id só é aceito se casar /^[A-Za-z0-9._:-]{1,128}$/; o valor '<b>x y' enviado foi substituído por UUID gerado (201 e nunca refletido). providerId/externalTransactionId continuam limitados a 1-128 chars e todo o SQL é parametrizado.	`src/interfaces/http/correlation-id.ts:3-7; src/application/wagering/wager-command.ts:22,34-35`
Ponto forte	Testes extras	Reverifica A-10 (aceito): mensagens de erro continuam em formato fechado {error:{code,message}}; erros inesperados viram 500 'internal error' sem stack no corpo.	`src/interfaces/http/http-error.filter.ts:76-97`
Ponto forte	Testes extras	Infra: portas do compose em 127.0.0.1, senhas de dev por env com default só local, Grafana anônimo apenas Viewer no profile observability, imagem como usuário bun, CI com actions fixadas por SHA e permissions contents: read, jose/helmet com versões fixas.	`docker-compose.yml:12,24,63,95; Dockerfile:12; .github/workflows/ci.yml:3-14; package.json:42-43`
Ponto forte	Chaves expostas	Reverifica B-01: todos os binds do compose estão em 127.0.0.1 (postgres, ministack, app, prometheus, jaeger, grafana). Confirmado por leitura do arquivo completo; nenhuma porta 'host:container' sem IP.	`docker-compose.yml:15,27,77,99,113,126`
Ponto forte	Chaves expostas	Reverifica B-02: init-roles.sql não tem senha literal; usa \getenv e :'var' (psql 15+). Os defaults owner/app existem só como ${VAR:-...} no compose e como fixture de teste.	`infra/postgres/init-roles.sql:9-13; docker-compose.yml:12-13`

Tipo	Categoria	O que está protegido	Evidência
Ponto forte	Chaves expostas	Reverifica B-03 (parcial, ver B-11 a B-13): a guarda existe, exige AUTH_MODE explícito em produção, recusa literalmente app:app, owner:owner, postgres:postgres e AWS_ACCESS_KEY_ID=test (testado, mensagens de erro nomeiam a variável e nunca repetem a senha), e o app inicia com AUTH desligada só com aviso.	`src/config/config.ts:96-104; src/interfaces/http/auth.module.ts:17-21`
Ponto forte	Chaves expostas	Reverifica B-05: o app não embute credenciais AWS (SDK usa a cadeia padrão); 'test'/'test' aparecem só no compose de dev, no compose de teste e em scripts/load-test.ts (emulador). O README avisa para usar IAM role.	`docker-compose.yml:39-40,71-72; scripts/load-test.ts:291,725; README.md:55`
Ponto forte	Chaves expostas	Reverifica B-09: Prometheus (127.0.0.1:9090) e Jaeger (127.0.0.1:16686) só publicam em loopback e estão no profile observability; imagens com tag fixa (prom/prometheus:v3.5.0, jaegertracing/all-in-one:1.62.0, grafana/grafana:12.1.0) e as tags existem (docker manifest inspect). Ver B-19 para o residual.	`docker-compose.yml:93,99,108,113,116,126`
Ponto forte	Chaves expostas	Histórico git completo (104 commits, git log --all) e 606d0dd..HEAD (59 commits): nenhum AKIA/ASIA, chave privada (BEGIN ... PRIVATE KEY), ghp_/github_pat_/xox/sk-, JWT (eyJ...), nem arquivo .env/.pem/.key (só .env.example) em qualquer commit. Os únicos acertos de regex são texto de relatório da rodada 1 descrevendo a busca e código de produção que contém a palavra 'token'.	`git log --all -p -G<padrões>; git log --all --name-only (.env.example, src/application/ports/tokens.ts, oidc-token-verifier.ts)`
Ponto forte	Chaves expostas	Chaves de teste do jose são geradas em memória a cada execução (generateKeyPair) e só a parte pública é exportada como JWK; nenhuma chave privada ou JWT de exemplo foi commitado, e o README só tem 'Authorization: Bearer ...' genérico.	`test/helpers/oidc.ts:15-17; README.md:200`
Ponto forte	Chaves expostas	Senhas em testes e fixtures (owner, app, postgres, 's3cr3t-Xk29', 'senha-forte', 'hunter2') são aceitáveis: ambiente efêmero em localhost:5433/4567, strings de teste para a guarda de config e para o filtro de erros (o teste http-error.filter prova que a senha não vaza). Nada aponta para host real.	`test/helpers/env.ts:14-17; test/unit/config/config-auth.test.ts:5,57-63; test/unit/interfaces/http-error.filter.test.ts:89`

Tipo	Categoria	O que está protegido	Evidência
Ponto forte	Chaves expostas	.env.example só traz valores de dev/emulador, com aviso de que os defaults valem somente para desenvolvimento local; .env fica fora do git e da imagem (ver B-14 para o limite do padrão).	`.env.example:1-13; .gitignore:2; .dockerignore:3`
Ponto forte	Chaves expostas	Reverifica B-08 (parte do USER): o Dockerfile roda como 'bun' (não root); código e node_modules ficam com dono root (instalados antes do USER), então o processo não consegue reescrever o próprio código. Sem ARG/ENV de segredo no Dockerfile.	`Dockerfile:5-11`
Ponto forte	Chaves expostas	Reverifica B-07: as duas actions estão fixadas por SHA e conferem com as tags declaradas (git ls-remote): actions/checkout 11d5960a3267... = refs/tags/v4.4.0; oven-sh/setup-bun 0c5077e51419... = refs/tags/v2.2.0. Runner ubuntu-24.04, permissions: contents: read, bun-version fixo.	`.github/workflows/ci.yml:4-5,11,14,16,18`
Ponto forte	Testes extras	Reverifica B-06: bun audit v1.4.4 retorna 'No vulnerabilities found (checked 333 packages)'. Os overrides ajv 8.20.0, js-yaml 5.5.0 e multer 2.5.0 existem no registro npm (multer/2.5.0 devolve HTTP 200; são as versões 'latest'), estão resolvidos no bun.lock; com esses overrides instalados o app sobe e /docs-json e /docs-yaml respondem 200 (testado ao vivo; não afirmo qual dependência usa o js-yaml).	`package.json:60-64; bun.lock:267,457,499; bun audit`
Ponto forte	Testes extras	Dependências do runtime sem pacotes de upload: multer (override) não é alcançado por nenhum controller; instalação com --frozen-lockfile no Docker e no CI.	`Dockerfile:6; .github/workflows/ci.yml:17`
Ponto forte	XSS	Reverifica B-10 (parcial confirmado): helmet ativo em todas as rotas, com X-Content-Type-Options: nosniff, X-Frame-Options, HSTS, Referrer-Policy e CSP restritivo (testado ao vivo em /health/live). Mensagens que ecoam input (nomes de chaves do zod; 'Cannot GET /%3Cscript%3E...' no 404) saem sempre em application/json com nosniff, então não são interpretáveis como HTML. X-Powered-By removido.	`src/main.ts:18,72-73; src/application/wagering/wager-command.ts:68-73; src/interfaces/http/http-error.filter.ts:72-76`
Ponto forte	XSS	Reverifica A-06/A-08 (limites): body de 70 KB recebeu 413 em JSON '{error:{code:PAYLOAD_TOO_LARGE}}' (testado), e JSON malformado recebe 400 INVALID_PAYLOAD, sem stack.	`src/main.ts:13,29-40,74-75`

Tipo	Categoria	O que está protegido	Evidência
Ponto forte	XSS	401 e 403 não ecoam input: o 401 sempre devolve a constante 'missing or invalid credentials' (testado com sem header, 'Bearer ', 'Bearer abc', token alg=none e 'Basic xx'), e o 403 usa mensagens constantes (scope X é constante do código; PROVIDER_MISMATCH fixa). O motivo da falha de JWT não vai ao cliente. X-Correlation-Id é validado por regex e nunca refletido.	`src/interfaces/http/auth.guard.ts:38-53; src/application/errors.ts:94-101; src/interfaces/http/correlation-id.ts:3-7`
Ponto forte	XSS	Verificador OIDC correto: issuer, audience e exp obrigatórios, algoritmos restritos a RS256/ES256 (bloqueia none/HS256 e confusão de chave), JWKS remoto com cache do jose, falha de rede vira 401. Token alg=none testado: 401.	`src/infrastructure/auth/oidc-token-verifier.ts:21-30`
Ponto forte	XSS	Apresentação: a tooltip usa tip.textContent (data-tip nunca vai para innerHTML); sem innerHTML, outerHTML, insertAdjacentHTML, document.write, eval, new Function, srcdoc, javascript:, onclick/onerror, iframe ou form; o único input externo é location.hash filtrado por /^#slide-(\d+)$/; os 4 links são relativos e estáticos; nenhum script ou CSS externo. Os setAttribute só recebem constantes ou números.	`docs/apresentacao/index.html:1000,1132,949-952,1096-1099`
Ponto forte	XSS	Gerador do relatório escapa todo texto do JSON: injetei '<script>alert(1)</script><img src=x onerror=alert(2)>"\'</td>' em id, title, file, lines, snippet, description, exploitability, impact, fix, acceptance, issueGroup, fixedIn, strengths, coverage, methodology e stack numa cópia do script e dos fixtures em diretório temporário (fora do repositório) e rodei com --sample: o HTML gerado tem 0 tags <script> e 0 <img>, e 10 ocorrências de '<script>alert(1)'. severity, category e status são validados contra listas fechadas antes de entrar em atributos style.	`docs/security-audit/generate-report.ts:124-137,153-170`
Ponto forte	XSS	Sem HTML dinâmico no backend: respostas são JSON ou text/plain; /docs só existe com DOCS_ENABLED (produção = 404 em /docs e /docs-json, testado na porta 4102).	`src/main.ts:77; src/config/config.ts:115`
Ponto forte	Chaves expostas	Auth novo (categoria 4): sem segredo no código OIDC; só URL do JWKS, issuer e audience vêm do ambiente. O logger faz redact de authorization e body.	`src/infrastructure/auth/oidc-token-verifier.ts:1-31; src/infrastructure/observability/logger.ts:9-12`

Pontos fracos (riscos centrais)

Nenhum achado crítico ou alto.

Achados por categoria

Categoria 1 — Banco sem tranca

Severidade	Status	Arquivo:linha	Descrição
Média	Aceito	`src/application/wagering/submit-wager-transaction.ts:93-108`	**A-R2-01 — Wallet não pertence a nenhum provider: qualquer provider autenticado movimenta e lê o saldo de qualquer wallet (resíduo de A-02)** Reverifica A-02. A correção de A-02 amarrou o providerId do body ao claim provider_id do token (confirmado, ver strengths), mas a wallet continua sem dono: o único vínculo checado é walletId + playerId (decide-wager.ts:43). Não existe relação provider -> wallet. Um provider com token válido consegue apostar/refundar em qualquer wallet cujo walletId e playerId conheça, e a resposta devolve o saldo resultante. Também há oráculo de existência: WALLET_NOT_FOUND (transactionId nulo) x WALLET_PLAYER_MISMATCH (com saldo) x PROCESSED.

Categoria 2 — Permissão definida no navegador

Nenhum achado nesta categoria.

Categoria 3 — IDOR

Nenhum achado nesta categoria.

Categoria 4 — Chaves expostas

Severidade	Status	Arquivo:linha	Descrição
Média	Corrigido `0035718`	`src/config/config.ts:79, 99-101`	**B-11 — Validação de produção da senha padrão é por substring literal e é contornada por URL-encoding, outro usuário ou query string** O bloqueio (reverifica B-03) procura as três substrings literais na DATABASE_URL crua. O driver pg decodifica a URL, então qualquer forma equivalente da mesma senha passa: 'postgres://wagering_app:%61pp@host/db', 'postgres://wagering_owner:%6Fwner@...', 'postgres://postgres:%70ostgres@...', 'postgres://wagering_owner@host/db?password=owner', e 'postgres://admin:postgres@...' (usuário diferente de 'postgres' com a senha padrão). Testei com loadConfig e também subindo o app de verdade: com NODE_ENV=production, AUTH_MODE=disabled e DATABASE_URL=postgres://wagering_app:%61pp@localhost:5433/..., o processo iniciou e /health/ready respondeu 'db=ok' (porta 4102 do TEST_PORT_BASE), ou seja, conectou em produção com a senha padrão 'app'. Rejeitados corretamente: app:app, owner:owner, postgres:postgres e o esquema postgresql://.

Severidade	Status	Arquivo:linha	Descrição
Média	Corrigido 0035718	`src/config/config.ts:88, 115`	**B-12 — NODE_ENV comparado por igualdade exata: 'Production', 'prod' ou 'production ' desligam toda a validação de produção e ligam o /docs** Qualquer valor diferente da string exata 'production' é tratado como desenvolvimento. Testado com loadConfig: NODE_ENV=Production, 'prod' e 'production ' (espaço) aceitam DATABASE_URL=postgres://postgres:postgres@db/w com AWS_ACCESS_KEY_ID=test, AUTH_MODE ausente, e devolvem docsEnabled=true e production=false. Falha aberta: o erro de grafia remove, de uma vez, a exigência de AUTH_MODE explícito, a recusa de senha/chave padrão, o aviso de auth desligada e o /docs fechado.
Baixa	Corrigido 0035718	`src/config/config.ts:102`	**B-13 — Guarda das chaves AWS cobre só AWS_ACCESS_KEY_ID === 'test' (grafia exata)** Testado: AWS_ACCESS_KEY_ID=' test' (espaço), 'TEST', AWS_SECRET_ACCESS_KEY=test sozinha, e chave ausente com SQS_ENDPOINT=http://localhost:4566 passam em produção. Também não há exigência de https no SQS_ENDPOINT nem checagem do AWS_SECRET_ACCESS_KEY. Se o SDK aceitará ' test' contra o MiniStack não foi testado; o ponto é que a guarda não o cobre.
Baixa	Corrigido b533745	`.gitignore:2`	**B-14 — .gitignore e .dockerignore só excluem '.env' exato: .env.production, .env.local e similares são versionados e entram na imagem** git check-ignore -v .env.production .env.local retorna vazio (rc=1): nenhum dos dois é ignorado. O Dockerfile faz 'COPY . .' (linha 8), então qualquer .env.* criado pelo operador para produção seria commitado por um 'git add .' e copiado para a imagem. A imagem também recebe test/ (com senhas de dev), docs/ e infra/, porque o .dockerignore só lista node_modules, .git, .env e dist. Reverifica B-02: o .env em si segue protegido.
Baixa	Aceito	`docker-compose.yml:119-124`	**B-16 — Reverifica B-04 (parcial): Grafana continua com admin/admin por default e ganhou acesso anônimo como Viewer** A senha deixou de ser literal, mas o default ${...:-admin} ainda resolve para a credencial padrão do Grafana, e o acesso anônimo (novo desde a rodada 1) deixa qualquer cliente que alcance a porta ver dashboards e métricas sem login. A porta está em 127.0.0.1 (confirmado em docker-compose.yml:126) e o README documenta isso.

Severidade	Status	Arquivo:linha	Descrição
Baixa	Corrigido 3fccb13	`src/config/config.ts:41-43`	**B-17 — OIDC_JWKS_URL e OIDC_ISSUER aceitam http:// em produção** z.string().url() aceita http://. Verificado: subi o app com NODE_ENV=production, AUTH_MODE=oidc e OIDC_JWKS_URL=http://127.0.0.1:4199/jwks (porta 4103) e ele iniciou sem objeção. As chaves públicas buscadas por HTTP puro podem ser trocadas por quem estiver no caminho de rede; o jose valida a assinatura contra o que o JWKS devolver. O verificador em si está correto (ver strengths).
Informativa	Aceito	`mikro-orm.config.ts:12, 39-41`	**B-21 — mikro-orm.config.ts mantém fallback owner:owner@localhost:5433 e cai para DATABASE_URL** A senha 'owner' é literal no config das migrations (aceitável: é o papel do docker-compose.test.yml, só localhost). O comando migration:up não passa por loadConfig, então nenhuma guarda de produção vale para ele. Com ambas as variáveis ausentes ele tenta o banco local em vez de falhar; com só DATABASE_URL usa o papel da aplicação, que não tem DDL (falha fechada).

Categoria 5 — XSS

Severidade	Status	Arquivo:linha	Descrição
Baixa	Corrigido b533745	`src/main.ts:16-23`	**B-15 — CSP desligado por inteiro em tudo que começa com '/docs', com justificativa que não confere** Verificado ao vivo: GET /health/live devolve o CSP default do helmet (script-src 'self', object-src 'none', frame-ancestors 'self'), e GET /docs, /docs-json e /docs-yaml saem sem Content-Security-Policy. O comentário diz que o Swagger usa script inline, mas o HTML servido em /docs só tem <script src=...> (swagger-ui-bundle.js, -standalone-preset.js, swagger-ui-init.js) e um <style> inline; style-src do helmet já permite 'unsafe-inline'. Não consegui renderizar em navegador, então não afirmo que o CSP padrão funcionaria; afirmo que a premissa do comentário não bate com o HTML. O prefixo também é amplo: qualquer rota futura que comece com '/docs' perde o CSP.

Categoria extra — Testes extras

Severidade	Status	Arquivo:linha	Descrição
Média	Corrigido 3fccb13	`src/infrastructure/auth/oidc-token-verifier.ts:22-26`	**A-R2-02 — Verificador JWT aceita token sem exp (token eterno)** jwtVerify só valida exp quando o claim existe; sem a opção requiredClaims: ["exp"], um JWT assinado corretamente e sem exp nunca expira. O doc-comment da classe diz que o verificador valida 'exp', o que não é verdade para a ausência do claim.

Severidade	Status	Arquivo:linha	Descrição
Média	Corrigido 3fccb13	`src/interfaces/http/auth.guard.ts:42-47`	**A-R2-03 — IdP fora do ar ou pendurado vira 401 (não 503) e cada requisição segura 5 s** Qualquer exceção do verify, inclusive falha de rede ao buscar o JWKS (JWKSTimeout, ECONNREFUSED), é convertida em 401 UNAUTHORIZED. O comportamento é fail-closed (ninguém passa), o que está correto, mas a indisponibilidade do IdP é indistinguível de credencial inválida. O esperado para esta auditoria era 503. Além disso, com o JWKS pendurado não há timeout menor que o padrão de 5 s do jose nem limite de concorrência: cada requisição autenticada espera 5 s.
Baixa	Corrigido b533745	`src/main.ts:73-75`	**A-R2-04 — Body parsing roda antes da autenticação: Content-Encoding inválido dá 500 INTERNAL e log de erro com stack para qualquer anônimo** O parser JSON é um middleware global que roda antes do AuthGuard. Só o erro entity.too.large é tratado em payloadTooLarge; os demais erros do body-parser caem no filtro global. Um corpo com Content-Encoding: gzip e bytes inválidos produz 500 INTERNAL e uma linha de log nível error com stack ('incorrect header check', zlibOnError). O gzip bomb em si está contido (limite aplicado após inflar: 413).
Baixa	Corrigido b533745	`src/interfaces/http/health.controller.ts:62-64`	**A-R2-05 — /health/ready e /metrics anônimos: readiness informa qual dependência caiu e dispara SELECT 1 + chamada SQS a cada hit** Reverifica A-07 (continua parcial, como documentado em ARCHITECTURE §16: rotas de infraestrutura para rede interna). O 503 de readiness devolve ao cliente 'not ready: db=down sqs=ok'. /metrics (metrics.controller.ts:5-17) e /health/* não têm guard. Não há label de providerId nem dado de negócio em /metrics (testado: nenhuma ocorrência de 'prov-').

Severidade	Status	Arquivo:linha	Descrição
Baixa	Corrigido 0035718	`src/config/config.ts:88; 96-100; 115`	**A-R2-06 — Validações de produção dependem de NODE_ENV=production exato; a imagem e o compose não o definem e a checagem de senha é por substring sensível a caixa** Todo o endurecimento novo (AUTH_MODE obrigatório, senhas padrão, AWS_ACCESS_KEY_ID=test, /docs desligado) só liga se NODE_ENV for exatamente 'production'. O Dockerfile (linhas 1-17) não define NODE_ENV e o docker-compose.yml não o define para o serviço app. Verificado com loadConfig: NODE_ENV=Production sem AUTH_MODE foi aceito; DATABASE_URL com ':APP@' (caixa diferente) passou; senha ':app1@' passou. AUTH_MODE=disabled em produção só gera um warn (config.ts / auth.module.ts:20-22). Em produção, DOCS_ENABLED=true liga o Swagger sem aviso.
Baixa	Corrigido 3fccb13	`src/config/config.ts:43`	**A-R2-07 — OIDC_JWKS_URL aceita qualquer esquema e host, inclusive http, file e IPs de metadados, também em produção** z.string().url() aceita http://, ftp://, file:/// e http://169.254.169.254/.... Verificado com loadConfig em NODE_ENV=production: todos 'ACCEPTED'. Não há exigência de https. O JWKS é a raiz de confiança de toda a autorização.
Baixa	Aceito	`src/main.ts:81`	**B-18 — A aplicação escuta em 0.0.0.0 e, rodando no host (bun run start), fica exposta na rede local com auth desligada** O bind em 127.0.0.1 do compose (B-01) só protege o caminho Docker. netstat durante o teste mostrou 0.0.0.0:4101, 0.0.0.0:4102 e 0.0.0.0:4103 em LISTENING. Com AUTH_MODE ausente (padrão fora de produção) todas as rotas, inclusive POST /wallets e a reconciliação, ficam abertas na LAN. O README fala em 'Segurança do ambiente local' só para o compose.
Informativa	Aceito	`src/infrastructure/auth/oidc-token-verifier.ts:17-19`	**A-R2-08 — Chave nova no JWKS só é aceita após o cooldown de 30 s do jose (401 temporário na rotação)** Padrões do jose: cooldownDuration 30 s e cacheMaxAge 10 min. Medido: depois de publicar a chave k2 no JWKS, um token com kid k2 foi 401 imediatamente e 200 após 31 s; 30 tokens com kid aleatório não geraram nenhum fetch extra (delta 0), ou seja, a proteção contra amplificação por kid desconhecido funciona. O cache em si funciona (5 requisições válidas, 0 fetch novo).

Severidade	Status	Arquivo:linha	Descrição
Informativa	Documentado	`src/interfaces/http/metrics.controller.ts:7-10`	**B-19 — Reverifica B-09 (residual): /metrics, /health/* e /docs ficam sem autenticação mesmo com AUTH_MODE=oidc** Verificado em produção com AUTH_MODE=oidc: /health/live e /metrics respondem 200 sem token (o AuthGuard só está nos controllers de wagering e wallets). /metrics expõe contadores por status/kind/source/component (prom-metrics.ts:13-65), sem identificadores de provedor ou de wallet. É a escolha esperada para scraping, e a correção de B-09 foi o bind local.
Informativa	Corrigido b533745	`README.md:56`	**B-20 — Documentação desatualizada: README diz que o startup não valida defaults, e o link do PDF na apresentação aponta para um arquivo movido** A validação existe desde b427a95 (config.ts:96-103), então a frase induz a achar que não há guarda (e, por outro lado, esconde que ela é parcial, ver B-11). Além disso docs/apresentacao/index.html:952 linka '../security-audit/relatorio-auditoria-seguranca.pdf', mas o PDF foi arquivado em docs/security-audit/round-1/ (commit 28762ab), então o link está quebrado.
Informativa	Aceito	`src/config/config.ts:57`	**B-22 — FAULT_INJECT=exit_after_commit é aceito em produção; imagens do compose fixadas por tag, sem digest** O gancho de falha (encerra o processo depois do commit) passa em NODE_ENV=production (testado). Exige controle do ambiente do processo, então só é risco de configuração. Em supply chain (reverifica B-08), todas as imagens têm tag fixa e existem (docker manifest inspect: prom/prometheus:v3.5.0, jaegertracing/all-in-one:1.62.0, grafana/grafana:12.1.0, postgres:17.6-alpine, amazon/aws-cli:2.17.0, ministackorg/ministack:1.5.35, oven/bun:1.4.4-alpine), mas nenhuma por digest, e tags podem ser reescritas no registry.
Informativa	Aceito	`docs/security-audit/generate-report.ts:596, 624`	**B-23 — Gerador do relatório abre a porta de depuração do Edge sem autenticação, em porta aleatória de loopback** Durante a geração, qualquer processo local da mesma máquina pode se conectar a 127.0.0.1:93xx e comandar o Edge (CDP), incluindo ler arquivos via file://. Janela curta, ferramenta de desenvolvimento, só loopback.

Recomendações priorizadas

P1 — crítica e alta: corrigir antes de qualquer exposição

Nenhum item.

P2 — média: corrigir no próximo ciclo

1. Média Aceito **A-R2-01** Wallet não pertence a nenhum provider: qualquer provider autenticado movimenta e lê o saldo de qualquer wallet (resíduo de A-02)
   Decisão do arquiteto: (a) modelar posse (wallets.provider_id ou tabela de vínculo provider-wallet/operator) e rejeitar com 403/422 uniforme quando o provider do token não é elegível para a wallet; ou (b) documentar que providers são confiáveis para qualquer wallet e remover o saldo da resposta de rejeições WALLET_PLAYER_MISMATCH (balance deve ser null quando player não confere), igualando WALLET_NOT_FOUND e WALLET_PLAYER_MISMATCH para não servir de oráculo.
2. Média Corrigido
   `3fccb13` **A-R2-02** Verificador JWT aceita token sem exp (token eterno)
   Passar requiredClaims: ["exp"] ao jwtVerify e, opcionalmente, maxTokenAge (ex.: "1h") e clockTolerance explícita e pequena (hoje é 0, o que é correto para o rigor, mas pode gerar 401 por skew de relógio).
3. Média Corrigido
   `3fccb13` **A-R2-03** IdP fora do ar ou pendurado vira 401 (não 503) e cada requisição segura 5 s
   Diferenciar erros do jose: JOSEError de verificação (JWTExpired, JWSSignatureVerificationFailed, JWTClaimValidationFailed, JWKSNoMatchingKey etc.) -> 401; JWKSTimeout/falha de fetch/ERR_JWKS_* -> TransientInfrastructureError (503 com Retry-After). Configurar timeoutDuration menor (ex.: 2000) em createRemoteJWKSet.
4. Média Corrigido
   `0035718` **B-11** Validação de produção da senha padrão é por substring literal e é contornada por URL-encoding, outro usuário ou query string
   Parsear a URL com new URL(), decodificar usuário e senha (decodeURIComponent) e ler também o parâmetro 'password' da query. Comparar a senha decodificada, sem depender do usuário, contra uma lista de senhas padrão (postgres, owner, app, admin, test, password). Preferir também exigir que a senha tenha comprimento mínimo em produção.
5. Média Corrigido
   `0035718` **B-12** NODE_ENV comparado por igualdade exata: 'Production', 'prod' ou 'production ' desligam toda a validação de produção e ligam o /docs
   Normalizar com trim().toLowerCase() e validar NODE_ENV como enum (development|test|production); valor desconhecido deve lançar. Calcular docsEnabled a partir de 'production', não de env.NODE_ENV cru.

P3 — baixa e informativa: melhoria e higiene

1. Baixa Corrigido
   `b533745` **A-R2-04** Body parsing roda antes da autenticação: Content-Encoding inválido dá 500 INTERNAL e log de erro com stack para qualquer anônimo
   No handler após o parser, mapear err.type 'encoding.unsupported', 'entity.parse.failed' e erros de zlib (statusCode 400/415) para 400/415 no formato {error:{code,message}} sem log nível error; ou rejeitar Content-Encoding diferente de identity (inflate: false no parser).
2. Baixa Corrigido
   `b533745` **A-R2-05** /health/ready e /metrics anônimos: readiness informa qual dependência caiu e dispara SELECT 1 + chamada SQS a cada hit
   Mensagem fixa 'not ready' no corpo (detalhe só no log), cache curto (1-2 s) do resultado de readiness, e orientar o deploy a expor /metrics e /health só na rede interna (ou bind separado/porta de admin).
3. Baixa Corrigido
   `0035718` **A-R2-06** Validações de produção dependem de NODE_ENV=production exato; a imagem e o compose não o definem e a checagem de senha é por substring sensível a caixa
   Definir ENV NODE_ENV=production no Dockerfile (e NODE_ENV=development explícito no compose de dev); normalizar com toLowerCase().trim(); comparar a senha parseando a URL (new URL(...).password) contra uma lista exata; tornar AUTH_MODE=disabled em production um erro, salvo variável explícita ALLOW_INSECURE_NO_AUTH=true.
4. Baixa Corrigido
   `3fccb13` **A-R2-07** OIDC_JWKS_URL aceita qualquer esquema e host, inclusive http, file e IPs de metadados, também em produção
   Em production exigir https: (new URL(v).protocol === 'https:'), rejeitar credenciais na URL e, se possível, listar hosts permitidos; http só fora de production (os testes usam http://127.0.0.1).
5. Baixa Corrigido
   `0035718` **B-13** Guarda das chaves AWS cobre só AWS_ACCESS_KEY_ID === 'test' (grafia exata)
   Comparar com trim().toLowerCase() e incluir AWS_SECRET_ACCESS_KEY; em produção exigir SQS_ENDPOINT https e não loopback/hostnames de compose, ou omitir SQS_ENDPOINT para usar o endpoint real da AWS.
6. Baixa Corrigido
   `b533745` **B-14** .gitignore e .dockerignore só excluem '.env' exato: .env.production, .env.local e similares são versionados e entram na imagem
   Usar '.env*' com exceção '!.env.example' em .gitignore e .dockerignore; acrescentar test, docs, .github e scripts ao .dockerignore.
7. Baixa Corrigido
   `b533745` **B-15** CSP desligado por inteiro em tudo que começa com '/docs', com justificativa que não confere
   Testar /docs com o CSP do helmet (ajustando só script-src/style-src/connect-src quando necessário) em vez de desligá-lo; restringir a correspondência a '/docs' e '/docs-json|yaml' exatos ou ao prefixo '/docs/'.
8. Baixa Aceito **B-16** Reverifica B-04 (parcial): Grafana continua com admin/admin por default e ganhou acesso anônimo como Viewer
   Sem default para GRAFANA_ADMIN_PASSWORD (${GRAFANA_ADMIN_PASSWORD:?defina no .env}) ou gerar senha aleatória no primeiro uso; manter anônimo desligado por padrão.

9. Baixa Corrigido
   `3fccb13` **B-17** OIDC_JWKS_URL e OIDC_ISSUER aceitam http:// em produção
   Em produção exigir https:// para OIDC_JWKS_URL (rejeitar o resto no loadConfig), com exceção opcional explícita para localhost em dev.
10. Baixa Aceito **B-18** A aplicação escuta em 0.0.0.0 e, rodando no host (bun run start), fica exposta na rede local com auth desligada
    Ler HOST do ambiente com default 127.0.0.1 fora do container; o compose define HOST=0.0.0.0 para o serviço app.
11. Informativa Aceito **A-R2-08** Chave nova no JWKS só é aceita após o cooldown de 30 s do jose (401 temporário na rotação)
    Documentar no runbook de rotação (publicar a nova chave no JWKS antes de começar a assinar com ela, com margem > 30 s). Se desejado, ajustar cooldownDuration.
12. Informativa Documentado **B-19** Reverifica B-09 (residual): /metrics, /health/* e /docs ficam sem autenticação mesmo com AUTH_MODE=oidc
    Bloquear /metrics na borda, ou servir em porta interna separada.
13. Informativa Corrigido
    `b533745` **B-20** Documentação desatualizada: README diz que o startup não valida defaults, e o link do PDF na apresentação aponta para um arquivo movido
    Atualizar o README para descrever o que a guarda cobre e o que não cobre; corrigir o href para round-1/ (ou para o relatório da rodada 2) e rodar bun run check:apresentacao.
14. Informativa Aceito **B-21** mikro-orm.config.ts mantém fallback owner:owner@localhost:5433 e cai para DATABASE_URL
    Falhar quando MIGRATION_DATABASE_URL e DATABASE_URL estiverem ausentes fora de NODE_ENV=test; mover o fallback para o script de teste.
15. Informativa Aceito **B-22** FAULT_INJECT=exit_after_commit é aceito em produção; imagens do compose fixadas por tag, sem digest
    Recusar FAULT_INJECT em produção; fixar imagens por digest (@sha256) no Dockerfile e no compose.
16. Informativa Aceito **B-23** Gerador do relatório abre a porta de depuração do Edge sem autenticação, em porta aleatória de loopback
    Usar --remote-debugging-pipe em vez de porta TCP, ou aceitar o risco e documentar.

ISSUES PARA O GITHUB

Cada bloco abaixo é uma issue completa em Markdown, somente para achados **aberto** ou **parcial**. Achados informativos sem ação não viram issue. Achados com o mesmo `issueGroup` ficam numa única issue; membros já corrigidos ficam fora dela.

Nenhum achado aberto ou parcial: nenhuma issue a abrir.

Já corrigidos

- `3fccb13` **A-R2-02** Verificador JWT aceita token sem exp (token eterno)
- `3fccb13` **A-R2-03** IdP fora do ar ou pendurado vira 401 (não 503) e cada requisição segura 5 s
- `0035718` **B-11** Validação de produção da senha padrão é por substring literal e é contornada por URL-encoding, outro usuário ou query string
- `0035718` **B-12** NODE_ENV comparado por igualdade exata: 'Production', 'prod' ou 'production ' desligam toda a validação de produção e ligam o /docs
- `b533745` **A-R2-04** Body parsing roda antes da autenticação: Content-Encoding inválido dá 500 INTERNAL e log de erro com stack para qualquer anônimo
- `b533745` **A-R2-05** /health/ready e /metrics anônimos: readiness informa qual dependência caiu e dispara SELECT 1 + chamada SQS a cada hit
- `0035718` **A-R2-06** Validações de produção dependem de NODE_ENV=production exato; a imagem e o compose não o definem e a checagem de senha é por substring sensível a caixa
- `3fccb13` **A-R2-07** OIDC_JWKS_URL aceita qualquer esquema e host, inclusive http, file e IPs de metadados, também em produção
- `0035718` **B-13** Guarda das chaves AWS cobre só AWS_ACCESS_KEY_ID === 'test' (grafia exata)
- `b533745` **B-14** .gitignore e .dockerignore só excluem '.env' exato: .env.production, .env.local e similares são versionados e entram na imagem
- `b533745` **B-15** CSP desligado por inteiro em tudo que começa com '/docs', com justificativa que não confere
- `3fccb13` **B-17** OIDC_JWKS_URL e OIDC_ISSUER aceitam http:// em produção
- `b533745` **B-20** Documentação desatualizada: README diz que o startup não valida defaults, e o link do PDF na apresentação aponta para um arquivo movido

Documentados (sem issue)

B-19 Reverifica B-09 (residual): /metrics, /health/* e /docs ficam sem autenticação mesmo com AUTH_MODE=oidc